

USE CASES / USER STORIES

Project: Smart Scientific Calculator

Development Model: Incremental Software Development Model

Course: Software Engineering & Project Management (SEPM)

1. Purpose

This document defines how users interact with the Smart Scientific Calculator. It translates the requirements identified in the SRS into user interactions, use cases, main flows, alternative flows, and acceptance criteria. The document is intended to support the subsequent system design, implementation, and testing activities.

2. System Actor

Actor	Description
User	The primary actor who enters numerical values, selects mathematical operations, views results, clears/deletes

3. Use Case Overview

ID	Use Case	Primary Actor	Increment
UC-01	Perform Addition	User	1
UC-02	Perform Subtraction	User	1
UC-03	Perform Multiplication	User	2
UC-04	Perform Division	User	2
UC-05	Perform Modulus / Percentage	User	2
UC-06	Perform Advanced Calculation	User	3
UC-07	Perform Scientific Calculation	User	4
UC-08	Clear / Delete Input	User	1–2
UC-09	View Calculation History	User	5
UC-10	Handle Invalid Operation	User	1–5

4. Use Case Diagram (Conceptual)

The system has one primary actor, the User. The User interacts with the calculator through the calculator interface. The following conceptual representation can be converted into a UML use-case diagram for the final submission.

USER	SMART SCIENTIFIC CALCULATOR
User	Perform Addition
User	Perform Subtraction
User	Perform Multiplication
User	Perform Division
User	Perform Modulus / Percentage
User	Perform Advanced Calculation

User	Perform Scientific Calculation
User	Clear / Delete Input
User	View Calculation History
User	Handle Invalid Operation

UML interpretation: one User actor is connected to the calculator use cases listed above. A graphical UML version should be included in the editable design artifact if required by the instructor.

UC-01 — Perform Addition

Primary Actor	User
Development Increment	Increment 1
Goal	Allow the user to add two numerical values.
Precondition	Calculator is open and ready for input.

Main Flow

1. User enters the first number.
2. User selects the addition (+) operation.
3. User enters the second number.
4. User selects the equals (=) operation.
5. System calculates the sum.
6. System displays the result.

Alternative / Exception Flow

- If an invalid numeric input is entered, the system requests/correctly handles valid input.

Postcondition

The calculated sum is displayed to the user.

Acceptance Criteria

For input $25 + 15$, the system displays 40.

UC-02 — Perform Subtraction

Primary Actor	User
Development Increment	Increment 1
Goal	Allow the user to subtract one numerical value from another.
Precondition	Calculator is open and ready for input.

Main Flow

1. User enters the first number.
2. User selects subtraction (-).
3. User enters the second number.
4. User selects (=).
5. System calculates the difference.
6. System displays the result.

Alternative / Exception Flow

- Invalid input is handled according to the system's error-handling rules.

Postcondition

The calculated difference is displayed.

Acceptance Criteria

For input 25 - 15, the system displays 10.

UC-03 — Perform Multiplication

Primary Actor	User
Development Increment	Increment 2
Goal	Allow the user to multiply numerical values.
Precondition	Calculator is open and ready for input.

Main Flow

1. User enters the first number.
2. User selects multiplication (×).
3. User enters the second number.
4. User selects (=).
5. System calculates the product.
6. System displays the result.

Alternative / Exception Flow

- Invalid input is handled appropriately.

Postcondition

The product is displayed.

Acceptance Criteria

For input 10×5 , the system displays 50.

UC-04 — Perform Division

Primary Actor	User
Development Increment	Increment 2
Goal	Allow the user to divide one numerical value by another.
Precondition	Calculator is open and ready for input.

Main Flow

1. User enters the dividend.
2. User selects division ($\div$).
3. User enters the divisor.
4. User selects (=).
5. System calculates the quotient.
6. System displays the result.

Alternative / Exception Flow

- If the divisor is zero, the system shall not perform the division and shall display an appropriate error.

Postcondition

Either the quotient or an appropriate error is displayed.

Acceptance Criteria

$20 \div 5$ displays 4; $20 \div 0$ displays an error.

UC-05 — Perform Modulus / Percentage

Primary Actor	User
Development Increment	Increment 2
Goal	Allow the user to perform remainder and percentage calculations.
Precondition	Calculator is open and ready for input.

Main Flow

1. User enters the required numerical values.
2. User selects modulus (%) or the percentage operation.
3. System evaluates the operation according to the defined calculator behaviour.
4. System displays the result.

Alternative / Exception Flow

- Invalid input or unsupported operation is handled appropriately.

Postcondition

The calculated result is displayed.

Acceptance Criteria

10 % 3 produces a remainder of 1 in modulus mode.

UC-06 — Perform Advanced Calculation

Primary Actor	User
Development Increment	Increment 3
Goal	Allow the user to perform square, power, square-root, and reciprocal operations.
Precondition	Calculator is open and the required numerical input is available.

Main Flow

1. User enters/selects the required value or values.
2. User selects an advanced operation.
3. System validates the input.
4. System performs the operation.
5. System displays the result.

Alternative / Exception Flow

- If the input is outside the valid domain, the system displays an appropriate error.

Postcondition

The calculated result or an appropriate error is displayed.

Acceptance Criteria

5^2 displays 25, 2^3 displays 8, and $\sqrt{25}$ displays 5.

UC-07 — Perform Scientific Calculation

Primary Actor	User
Development Increment	Increment 4
Goal	Allow the user to perform sine, cosine, tangent, logarithm, and natural logarithm operations.
Precondition	Calculator is open and scientific functions are available.

Main Flow

1. User enters the required value.
2. User selects a scientific operation.
3. System validates the input and relevant mode.
4. System performs the calculation.
5. System displays the result.

Alternative / Exception Flow

- For an input outside the mathematical domain, the system displays an appropriate error.

Postcondition

The scientific result or an appropriate error is displayed.

Acceptance Criteria

$\sin(0)$ produces 0, subject to the selected angle mode and numerical precision.

UC-08 — Clear / Delete Input

Primary Actor	User
Development Increment	Increment 1–2
Goal	Allow the user to remove current input or clear the current calculation.
Precondition	The calculator contains user-entered input.

Main Flow

1. User selects Clear to reset the current calculation, or Delete/Backspace to remove the most recent input character where applicable.
2. System updates the display.

Alternative / Exception Flow

- If there is no input to delete, the system maintains a valid empty/current state.

Postcondition

The calculator display reflects the requested clear/delete action.

Acceptance Criteria

After entering 123, Delete removes the last entered character; Clear resets the current calculation.

UC-09 — View Calculation History

Primary Actor	User
Development Increment	Increment 5
Goal	Allow the user to review previously performed calculations.
Precondition	At least one calculation has been performed and stored in history.

Main Flow

1. User opens the history view.
2. System retrieves stored calculation entries.
3. System displays previous expressions and their results.

Alternative / Exception Flow

- If no history exists, the system displays an appropriate empty-history message/state.

Postcondition

The user can view available calculation history.

Acceptance Criteria

After performing $25 + 15 = 40$, the corresponding calculation can be viewed in history.

UC-10 — Handle Invalid Operation

Primary Actor	User
Development Increment	Increment 1–5
Goal	Prevent invalid mathematical operations from producing misleading results.
Precondition	User attempts an operation using an invalid input or invalid mathematical domain.

Main Flow

1. User enters an invalid operation or input.
2. System validates the operation.
3. System detects the invalid condition.
4. System displays an appropriate error message.

Alternative / Exception Flow

- The user can correct the input and attempt a valid calculation.

Postcondition

The invalid operation is not silently accepted as a valid result.

Acceptance Criteria

Division by zero results in an error rather than a normal numerical result.

5. User Stories

ID	User Story	Acceptance Criteria	Priority
US-01	As a user, I want to add two numbers so that I can perform basic calculations.	Given two valid numbers, selecting + and = displays the sum.	Must Have.
US-02	As a user, I want to subtract two numbers so that I can calculate the difference.	Given two valid numbers, selecting - and = displays the difference.	Must Have.
US-03	As a user, I want multiplication and division so that I can perform operations and display correct results; division by zero is rejected.	Valid perform operations and display correct results; division by zero is rejected.	Must Have.
US-04	As a user, I want modulus and percentage operations so that I can perform calculations and produce the defined results.	These operations perform and produce the defined results.	Should Have.
US-05	As a user, I want square, power, square-root and reciprocal operations so that I can perform advanced operations.	Calculation produces correct advanced operations results.	Should Have.
US-06	As a user, I want scientific functions so that I can perform scientific calculations.	Valid results for supported scientific calculations.	Should Have.
US-07	As a user, I want to clear or delete input so that I can reset the current calculation and remove the applicable last input.	Clear resets the current calculation and Delete removes the applicable last input.	Must Have.
US-08	As a user, I want to view calculation history so that I can review previously stored calculations.	Previously stored calculations displayed in the history.	Should Have.
US-09	As a user, I want invalid operations to produce clear error messages such as division by zero.	Invalid operations such as division by zero display an appropriate error.	Must Have.

6. Traceability to SRS

SRS Requirement Group	Related Use Cases
FR-01, FR-02	UC-01, UC-02
FR-03, FR-04	UC-03, UC-04
FR-05, FR-06	UC-05
FR-07 to FR-10	UC-06
FR-11 to FR-15	UC-07
FR-16, FR-17	UC-08
FR-18	UC-09
FR-19	UC-10
FR-20	All calculation use cases

Note: The use cases in this document are derived from the current SRS scope. If the SRS is formally changed later, the affected use cases and acceptance criteria should also be updated to maintain requirements traceability.